

# Penalised regression

Machine Learning in Finance · Session 6

## Last time: choosing between candidates

---

**0.740**

one column

**0.772**

two columns

**0.747**

three columns

Cross-validation compared three candidates and kept the one-column model. Each candidate either contained a column or did not.

Three columns give eight possible subsets. A real table has far more columns than three.

# This session

- a table with **nineteen** columns, and what plain regression does with it
- a **penalty** on the size of the coefficients: ridge regression
- why the penalty needs every column on the same **scale**, and a pipeline
- choosing the penalty's strength with the folds: **hyperparameter tuning**
- the **lasso**, which sets coefficients to exactly zero, and the elastic net
- what the arguments of these scikit-learn objects do, and when to change them

---

# 1 • Many columns

## PART 1

A wider table, and what ordinary least squares does with it.

# The question

**How volatile will the whole market be over the next twenty trading days?**

## Why it matters

the risk of almost any portfolio moves with the index, so its volatility sets margins, limits and option prices

## What you know today

the index's own history over several windows, and how volatile each stock in it has been lately

# The table

Python Code

```
table = pd.read_csv("data/market_features.csv", parse_dates=["date"]).set_index("date")
```

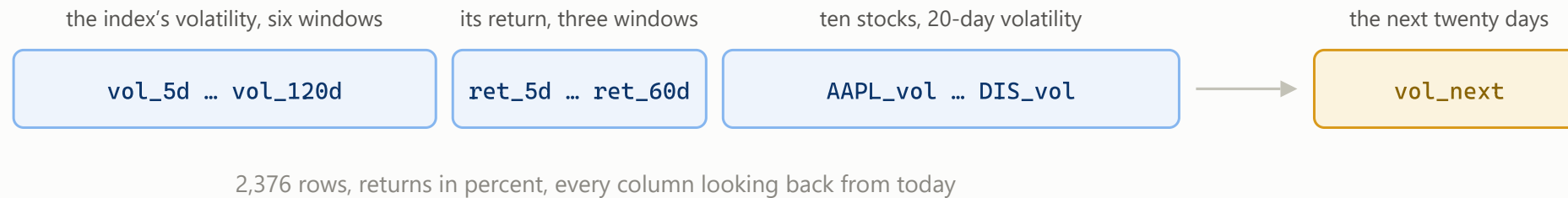
```
print(table.shape)  
print(list(table.columns))
```

```
(2376, 20)
```

```
['vol_5d', 'vol_10d', 'vol_20d', 'vol_40d', 'vol_60d', 'vol_120d', 'ret_5d', 'ret_20d',  
'ret_60d', 'AAPL_vol', 'MSFT_vol', 'NVDA_vol', 'JPM_vol', 'KO_vol', 'PG_vol', 'XOM_vol',  
'JNJ_vol', 'WMT_vol', 'DIS_vol', 'vol_next']
```

One row per trading day, built from `prices.csv` with `rolling` and `shift` as last time. The index is SPY, the S&P 500 fund in the data.

# Nineteen columns, one target



Building this table from the price file is one of the exercises.

# Splitting by date

Python Code

```
train = table.loc[:"2022-12-31"]
test = table.loc["2023-01-01":]

columns = list(table.columns[:-1])
print(len(train), len(test), len(columns))
```

```
1894 482 19
```

`table.columns[:-1]` is every column except the last, which is the target.

# A helper for the error

Python Code

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

def rmse(actual, predicted):
    return np.sqrt(mean_squared_error(actual, predicted))
```

The RMSE is computed many times this session. A two-line function keeps every later cell short.

# One column, as before

Python Code

```
one = LinearRegression()  
one.fit(train[["vol_20d"]], train["vol_next"])  
  
print("train:", rmse(train["vol_next"], one.predict(train[["vol_20d"]]])))  
print("test :", rmse(test["vol_next"], one.predict(test[["vol_20d"]]])))
```

```
train: 0.5877315830394847  
test : 0.2369234893264645
```

This is the same model as last time, on the index. A test error of 0.237 is the number to beat.

# All nineteen columns

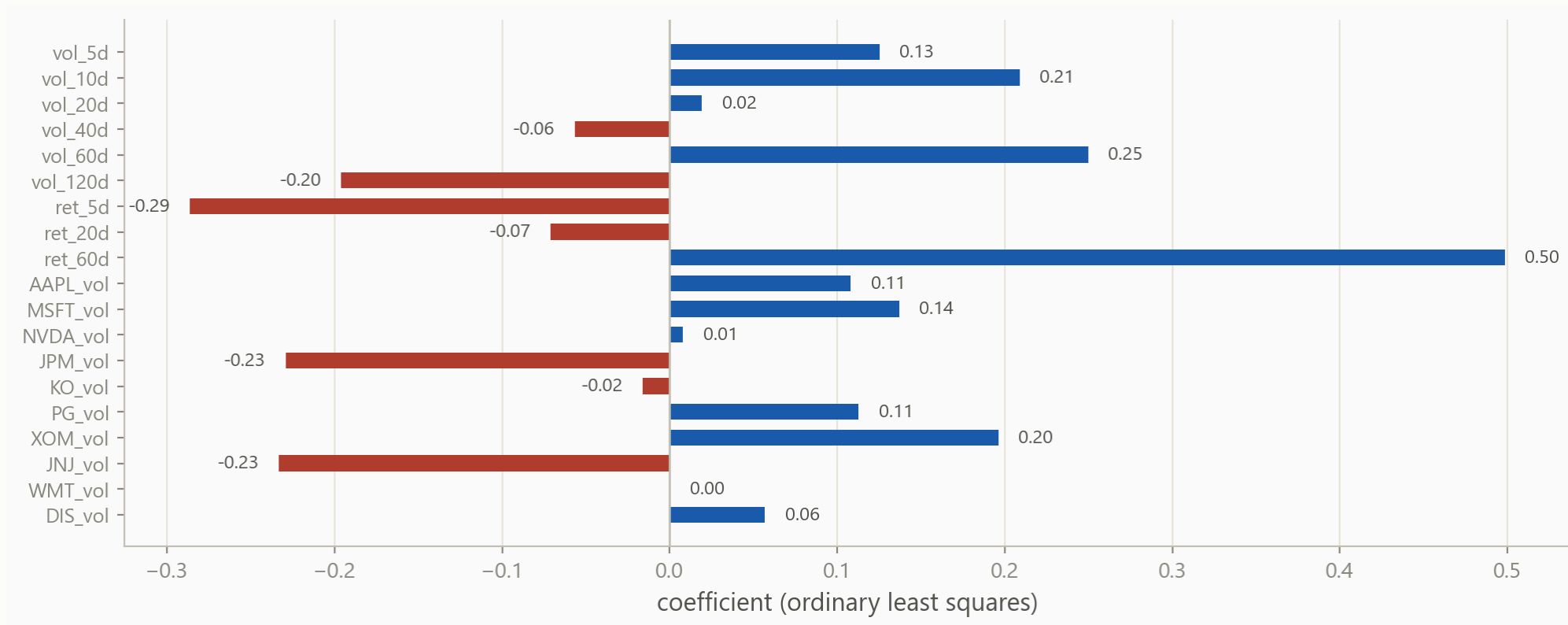
Python Code

```
ols = LinearRegression()  
ols.fit(train[columns], train["vol_next"])  
  
print("train:", rmse(train["vol_next"], ols.predict(train[columns])))  
print("test :", rmse(test["vol_next"], ols.predict(test[columns])))
```

```
train: 0.5199292578787686  
test : 0.248148741760285
```

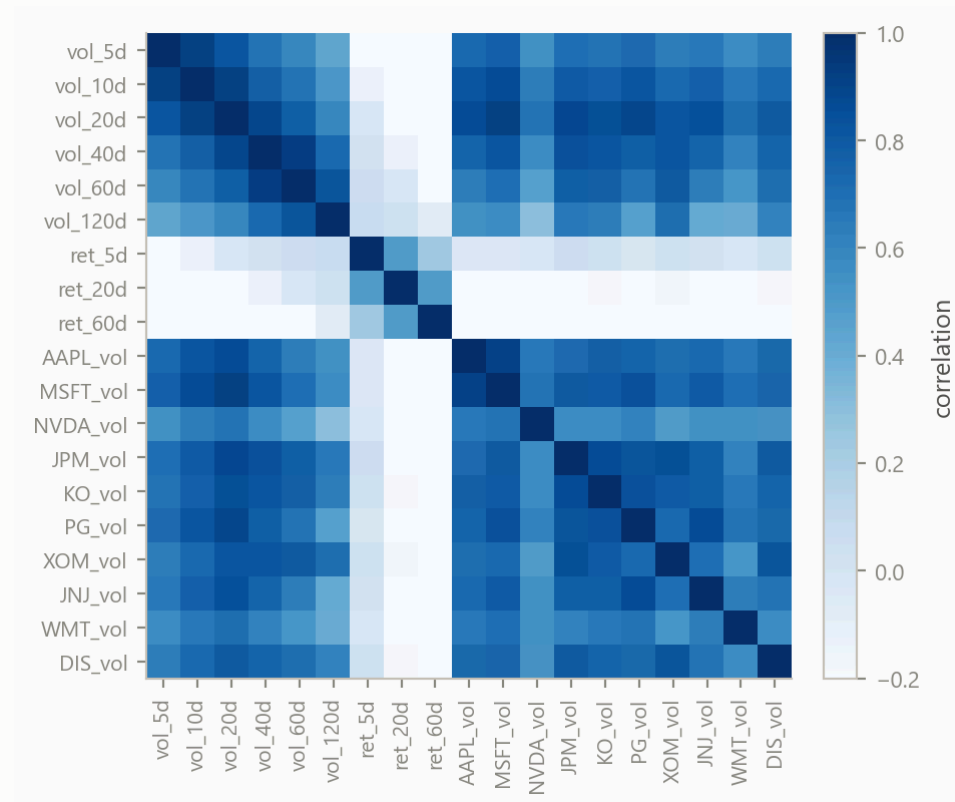
The training error fell and the test error rose. Eighteen extra columns made the forecast worse.

# The nineteen coefficients



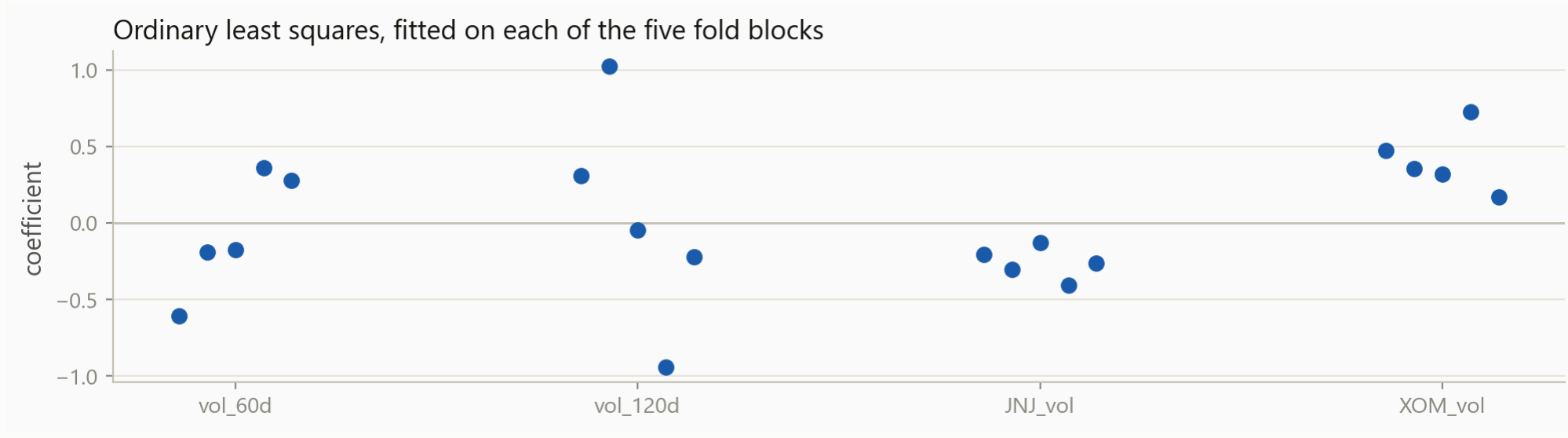
Higher volatility at J&J and JPMorgan **lowers** the forecast, and the index's own volatility raises it at 60 days and lowers it at 120. Those signs are not believable.

# The columns are near-copies of each other



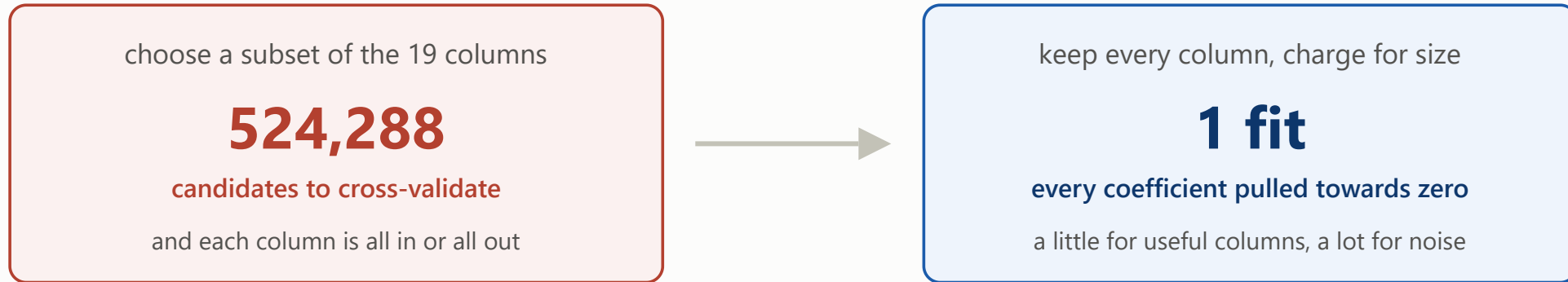
**vol\_40d** and **vol\_60d** correlate at 0.93. Two columns with the same information can take +5 and -4.8 and fit exactly as well as +0.1 and +0.1. The fit is fine; the coefficients are noise.

# The coefficients depend on the rows



Each column has five dots, one per block of training rows. **vol\_120d** is +1.02 on one block and  $-0.94$  on another. A coefficient that changes sign with the sample cannot be used for next year.

# From half a million fits to one



the right-hand side is penalised regression, also called regularised

Subset selection asks, for each column, in or out. A penalty asks a softer question: **how large may its coefficient be?**

---

## 2 · Ridge regression

### PART 2

A penalty on the squared coefficients, and one new argument.

# What ordinary least squares minimises

$$\text{RSS} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

The residual sum of squares on the training rows, and nothing else. A coefficient of +5 is accepted if it lowers the RSS by a little, which is how +5 and −4.8 on two near-copies come about.

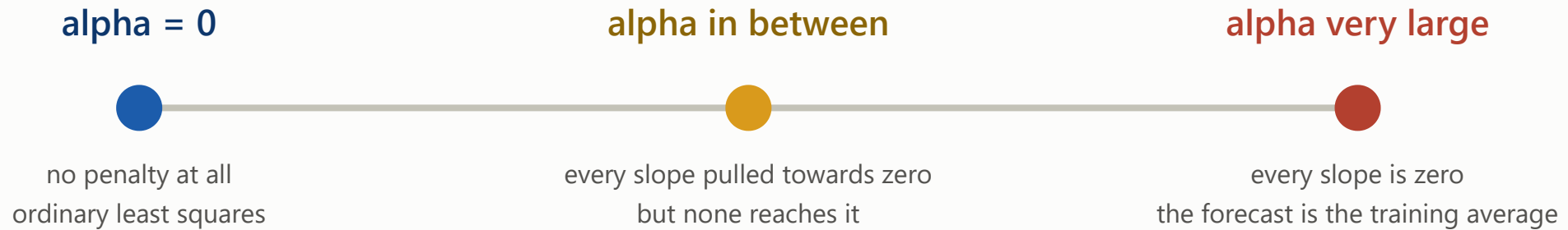
# The ridge objective

$$\text{RSS} + \alpha \sum_{j=1}^p \beta_j^2$$

Ridge adds a charge for the size of the slopes. A coefficient of 5 now costs 25 times **alpha**, and is kept only if it lowers the RSS by more than that.

The sum runs over the slopes and leaves the intercept alone.

# What alpha does



`alpha` is the price of coefficient size. It is set before `.fit()` runs, and the rows cannot choose it, because every value gives a fitted model.

# Ridge in code

Python Code

```
from sklearn.linear_model import Ridge

ridge = Ridge(alpha=1000)
ridge.fit(train[columns], train["vol_next"])

print(rmse(test["vol_next"], ridge.predict(test[columns])))
```

```
0.22838911600446554
```

Only the first line changed: **Ridge** fits and predicts exactly like **LinearRegression**.  
0.228, against 0.248 unpenalised and 0.237 for one column.

## Your turn 1

---

Fit the same nineteen columns with a much smaller penalty and a much larger one. Which of the earlier numbers does each land near?

Python Code

```
small = Ridge(alpha=...)
small.fit(train[columns], train["vol_next"])
print("alpha small:", rmse(test["vol_next"], small.predict(test[columns])))

large = Ridge(alpha=...)
large.fit(train[columns], train["vol_next"])
print("alpha large:", rmse(test["vol_next"], large.predict(test[columns])))
```

Try 1 for the small one and 100000 for the large one.

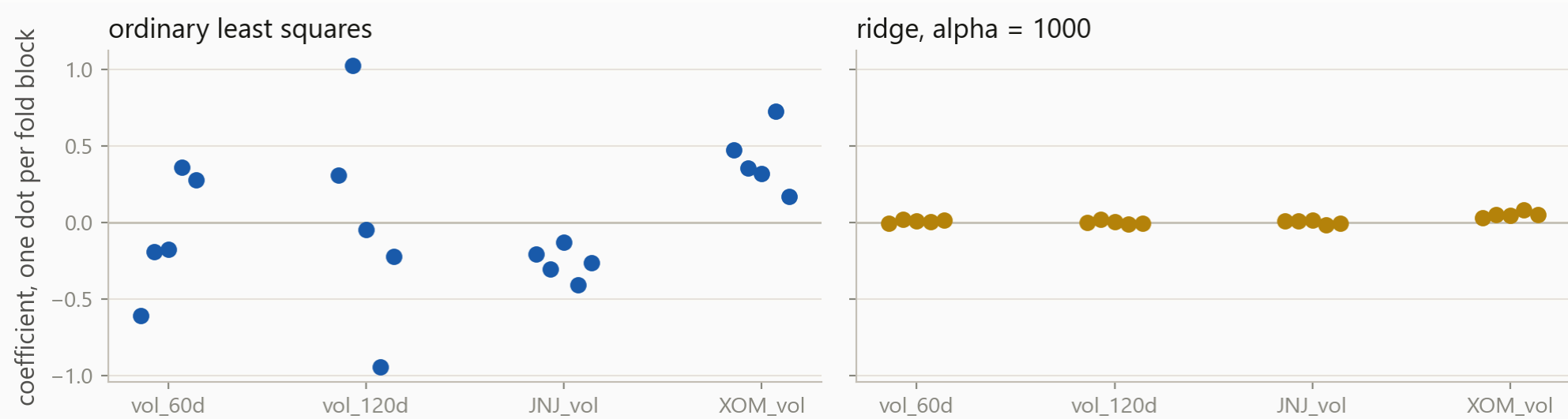
## Your turn 1: answer

```
small = Ridge(alpha=1)           # 0.247  
large = Ridge(alpha=100000)      # 0.282
```

**alpha=1** is ordinary least squares to three decimals: a penalty of 1 is nothing next to an RSS over 1,894 rows. **alpha=100000** is worse than repeating this month, and at ten million the error is 0.291, the same as guessing the average.

Too little penalty leaves the overfitting in. Too much removes the signal with the noise.

# Why smaller coefficients forecast better



These are the same four columns on the same five blocks. The ridge coefficients are small and they stay put, so a coefficient fitted on the past holds on the years to come. That is a little bias in exchange for far less variance.

---

# 3 • The penalty and the units

## PART 3

The penalty charges by coefficient size, and size depends on units.

# The same column, in two units

|                     | typical values       | the OLS coefficient | its ridge penalty, $\beta^2$ |
|---------------------|----------------------|---------------------|------------------------------|
| vol_20d in percent  | 0.98 1.21 0.90       | <b>0.019</b>        | <b>0.0004</b>                |
| vol_20d in decimals | 0.0098 0.0121 0.0090 | <b>1.913</b>        | <b>3.66</b>                  |

the same column, the same forecast, and a penalty ten thousand times larger

Ridge shrinks the column in small units first, whatever it contains. The penalty is judging columns by their units, not by their usefulness.

# Measured: OLS

Python Code

```
train_dec = train.copy()
test_dec = test.copy()
train_dec["vol_20d"] = train_dec["vol_20d"] / 100
test_dec["vol_20d"] = test_dec["vol_20d"] / 100

spot = columns.index("vol_20d")

ols_dec = LinearRegression()
ols_dec.fit(train_dec[columns], train_dec["vol_next"])
print(ols.coef_[spot], ols_dec.coef_[spot])
print(rmse(test["vol_next"], ols.predict(test[columns])), rmse(test["vol_next"], ols_dec.predict(test_dec[columns])))
```

0.019130121486974225 1.9130121486975642

0.248148741760285 0.24814874176028537

# Measured: ridge

Python Code

```
ridge_dec = Ridge(alpha=1000)
ridge_dec.fit(train_dec[columns], train_dec["vol_next"])

print(ridge.coef_[spot], ridge_dec.coef_[spot])
```

```
0.031412518408288924 0.0003499520203564534
```

The coefficient on **vol\_20d** falls from 0.031 to 0.0003. Ridge has dropped the column, and nothing about its information changed.

# Put every column on the same scale

$$z = \frac{x - \bar{x}}{s}$$

Subtract each column's mean and divide by its standard deviation. Every column then has mean 0 and standard deviation 1, whatever units it arrived in, and one coefficient is comparable with another.

The standardising formula from two sessions ago, where it was noted that penalised regression needs it.

# StandardScaler

Python Code

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaler.fit(train[columns])
X_train = scaler.transform(train[columns])
X_test = scaler.transform(test[columns])

print(scaler.mean_[spot], scaler.scale_[spot])
print(round(X_train[:, spot].mean(), 6), round(X_train[:, spot].std(), 6))
```

```
0.978877970095655 0.6927602187417808
-0.0 1.0
```

**.fit()** learns each column's mean and standard deviation from the training rows.  
**.transform()** applies them, to the test rows as well: their own mean is information from the future.

# Ridge on standardised columns

Python Code

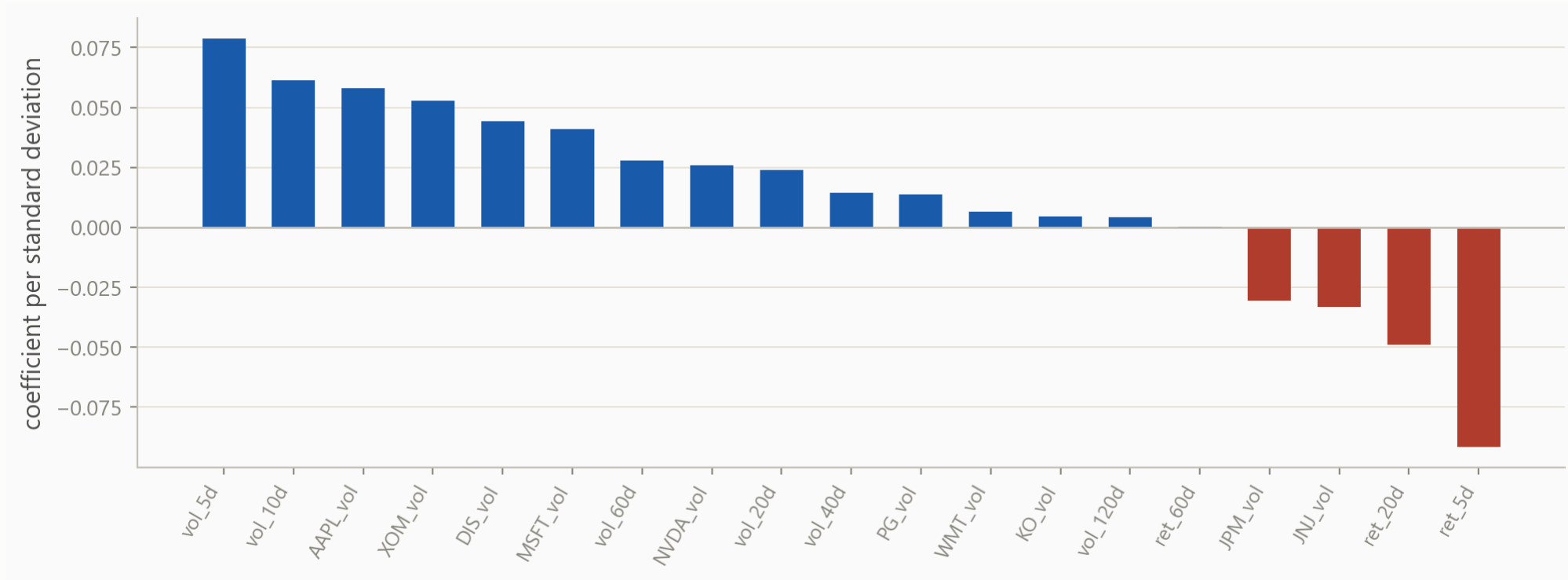
```
ridge = Ridge(alpha=1000)
ridge.fit(X_train, train["vol_next"])

print(rmse(test["vol_next"], ridge.predict(X_test)))
```

0.21808004018582344

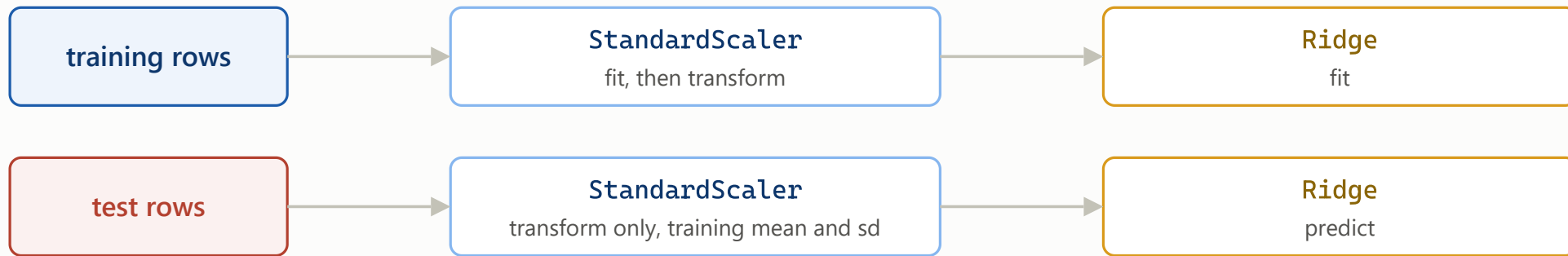
With every column in standard deviations, the same model and the same alpha score 0.218, down from 0.228.

# Reading standardised coefficients



The bars are comparable now. The last five and ten days matter most, a market that fell in the last week is more volatile in the month after, and the other stocks each add a little.

# Two steps that belong together



The training rows are scaled and then fitted; the test rows are scaled the same way and then predicted. Cross-validation repeats the pair on five fitting blocks, so the scaler has to be refitted inside every fold with the model.

# Pipeline

Python Code

```
from sklearn.pipeline import Pipeline

pipe = Pipeline([("scale", StandardScaler()), ("ridge", Ridge(alpha=1000))])
pipe.fit(train[columns], train["vol_next"])

print(rmse(test["vol_next"], pipe.predict(test[columns])))
```

0.21808004018582344

A pipeline is a list of named steps. `.fit()` fits the scaler, transforms, and fits the ridge; `.predict()` transforms with the fitted scaler and predicts. The names are yours to choose.

# A pipeline is a model

Python Code

```
from sklearn.model_selection import TimeSeriesSplit, cross_val_score

folds = TimeSeriesSplit(n_splits=5)

scores = cross_val_score(pipe, train[colums], train["vol_next"],
                          cv=folds, scoring="neg_root_mean_squared_error")
print(-scores.round(3))
print(-scores.mean())
```

```
[0.476 0.472 1.127 0.327 0.397]
0.5599413071612366
```

`cross_val_score` takes the pipeline where it took `LinearRegression()` last time, and refits the scaler inside each fold. The third fold still contains March 2020.

---

# 4 • Choosing alpha

## PART 4

A setting the data cannot pick, and the tool that searches over it.

# Parameters and hyperparameters

## Parameters

The intercept and the nineteen slopes. `.fit()` computes them from the training rows and stores them with a trailing underscore.

## Hyperparameters

`alpha`. Passed in before `.fit()` and never changed by it. Every value gives a fitted model, so the training rows cannot say which value is right.

A hyperparameter is chosen the way candidates were chosen last time: fit with each value, score on held-out rows, keep the best.

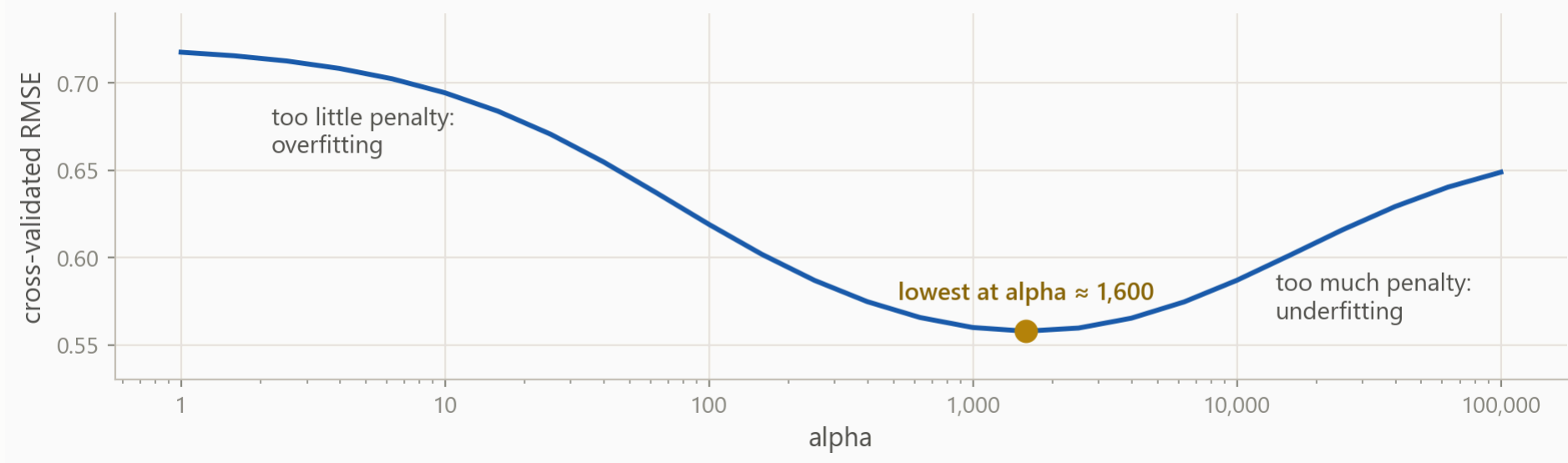
# Try several

Python Code

```
for alpha in [1, 10, 100, 1000, 10000]:  
    pipe = Pipeline([("scale", StandardScaler()), ("ridge", Ridge(alpha=alpha))])  
    scores = cross_val_score(pipe, train[colums], train["vol_next"],  
                             cv=folds, scoring="neg_root_mean_squared_error")  
    print(alpha, round(-scores.mean(), 4))
```

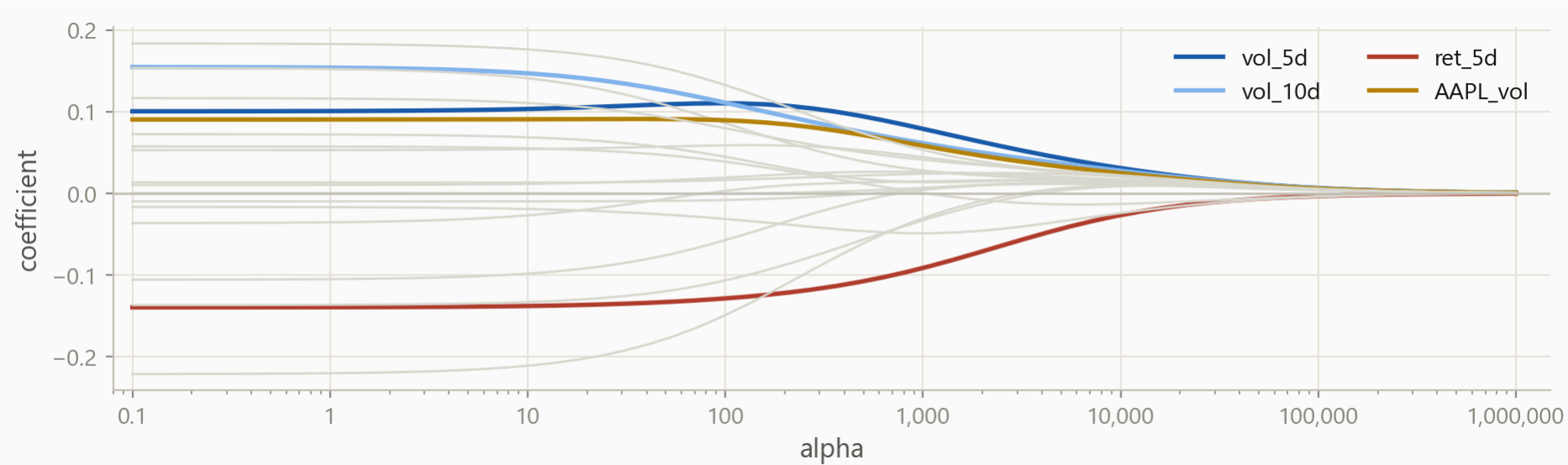
```
1 0.7176  
10 0.6944  
100 0.619  
1000 0.5599  
10000 0.587
```

# The validation curve



This is the same loop over a finer grid. High on the left where the penalty is too weak, high on the right where it is too strong, lowest at a few thousand.

# Why the grid goes in factors of ten



Each line is one coefficient as alpha grows. Between 1 and 10 almost nothing happens; between 1,000 and 10,000 everything does. The grid steps by a factor, not by a fixed amount.

# GridSearchCV

|             | fold 1 | fold 2 | fold 3 | fold 4 | fold 5 | mean  |
|-------------|--------|--------|--------|--------|--------|-------|
| alpha 1     | 0.554  | 0.653  | 1.078  | 0.890  | 0.414  | 0.718 |
| alpha 10    | 0.537  | 0.628  | 1.077  | 0.823  | 0.408  | 0.694 |
| alpha 100   | 0.540  | 0.534  | 1.082  | 0.547  | 0.392  | 0.619 |
| alpha 1000  | 0.476  | 0.472  | 1.127  | 0.327  | 0.397  | 0.560 |
| alpha 10000 | 0.472  | 0.481  | 1.236  | 0.283  | 0.463  | 0.587 |

**best** **alpha = 1000** has the lowest mean. It is refitted on all 1,894 training rows, and that model is the result.

Five values times five folds is twenty-five fits, run by one call.

# GridSearchCV in code

Python Code

```
from sklearn.model_selection import GridSearchCV

pipe = Pipeline([("scale", StandardScaler()), ("ridge", Ridge())])
grid = {"ridge__alpha": [1, 10, 100, 1000, 10000]}

search = GridSearchCV(pipe, grid, cv=folds, scoring="neg_root_mean_squared_error")
search.fit(train[columns], train["vol_next"])

print(search.best_params_)
print(-search.best_score_)
```

```
{'ridge__alpha': 1000}
0.5599413071612366
```

The grid is a dictionary of settings and the values to try. **"ridge\_\_alpha"** is the step name, two underscores, then the argument: that is how a setting inside a pipeline is addressed.

# What the search holds afterwards

Python Code

```
results = pd.DataFrame(search.cv_results_)
results[["param_ridge__alpha", "mean_test_score", "std_test_score", "rank_test_score"]]
```

|   | param_ridge__alpha | mean_test_score | std_test_score | rank_test_score |
|---|--------------------|-----------------|----------------|-----------------|
| 0 | 1                  | -0.717629       | 0.237574       | 5               |
| 1 | 10                 | -0.694355       | 0.234213       | 4               |
| 2 | 100                | -0.619026       | 0.238827       | 3               |
| 3 | 1000               | -0.559941       | 0.288848       | 1               |
| 4 | 10000              | -0.586998       | 0.332713       | 2               |

`cv_results_` is the whole grid as a table: every value tried, its mean score over the folds, the spread, and its rank.

# Predicting with the search

Python Code

```
print(search.best_estimator_)  
print(rmse(test["vol_next"], search.predict(test[columns])))
```

```
Pipeline(steps=[('scale', StandardScaler()), ('ridge', Ridge(alpha=1000))])  
0.21808004018582344
```

`best_estimator_` is the winning pipeline refitted on every training row, and `search.predict()` uses it. The test rows are opened once, here, and played no part in choosing alpha.

# The arguments of GridSearchCV

| argument                        | what it changes                     | when to change it                                          |
|---------------------------------|-------------------------------------|------------------------------------------------------------|
| <code>estimator</code>          | the model or pipeline being tuned   | always a pipeline when scaling is involved                 |
| <code>param_grid</code>         | the settings and the values to try  | keep each list short; several keys multiply                |
| <code>cv</code>                 | how the folds are made              | always set it: the default ignores time order              |
| <code>scoring</code>            | the score, larger is better         | match it to the error you will report                      |
| <code>refit</code>              | refit the best on all training rows | leave <code>True</code> , so <code>.predict()</code> works |
| <code>n_jobs</code>             | how many folds run at once          | <code>-1</code> on your own machine, to use every core     |
| <code>return_train_score</code> | also record the training score      | to see how large the overfitting gap is                    |

## Your turn 2

---

The best alpha was 1000, with 100 and 10000 on either side. Search a finer grid around it and see whether the score improves.

Python Code

```
fine = {"ridge__alpha": [...]}

fine_search = GridSearchCV(pipe, fine, cv=folds, scoring="neg_root_mean_squared_error")
fine_search.fit(train[columns], train["vol_next"])

print(fine_search.best_params_)
print(-fine_search.best_score_)
```

Five values between 300 and 4000, stepping by roughly a factor of two.

## Your turn 2: answer

```
fine = {"ridge__alpha": [300, 600, 1000, 2000, 4000]}
```

The best is **2000**, at 0.5583 against 0.5599 for 1000. On the test rows the two give 0.217 and 0.218.

The gain is in the third decimal, and the fold scores range from 0.33 to 1.13. One significant figure of alpha is enough.

---

# 5 • Lasso

## PART 5

A different penalty, which sets coefficients to exactly zero.

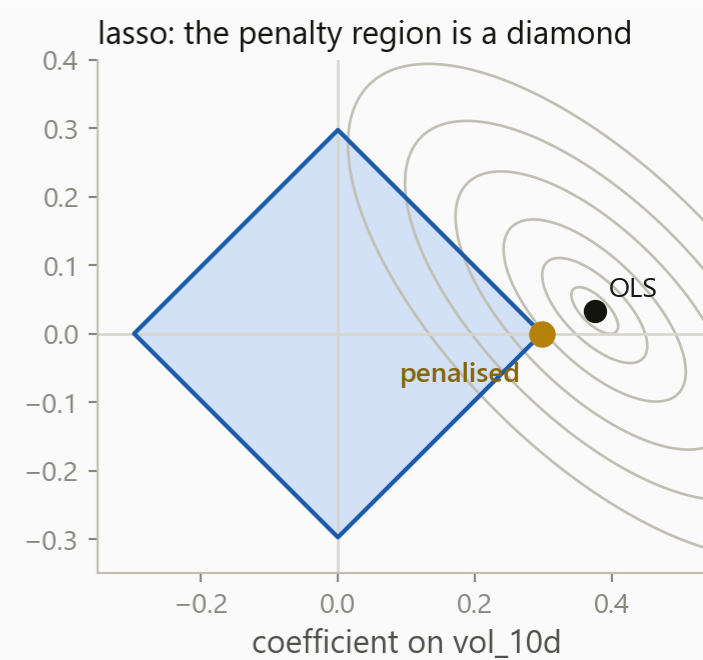
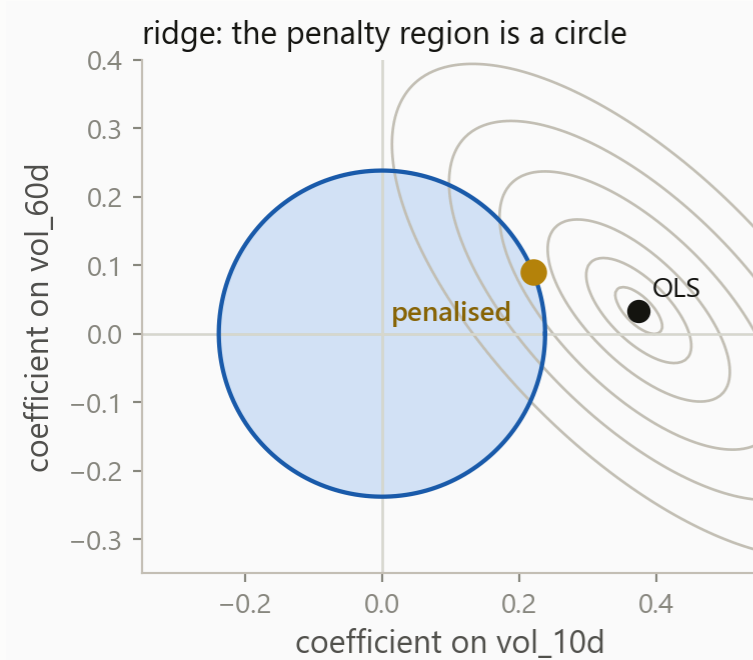
# One change to the penalty

$$\text{ridge: } \text{RSS} + \alpha \sum_{j=1}^p \beta_j^2$$

$$\text{lasso: } \text{RSS} + \alpha \sum_{j=1}^p |\beta_j|$$

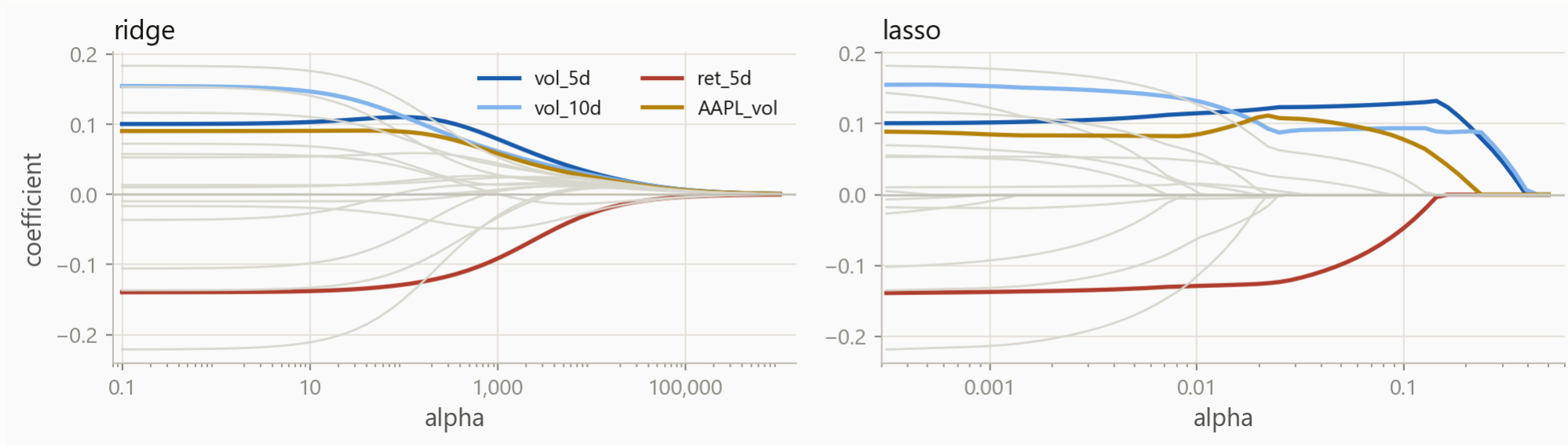
The squares become absolute values. Everything else is the same: a price on size, set by alpha, chosen by cross-validation.

# Why absolute values give exact zeros



With two columns the fit can be drawn. The grey rings are equal RSS around the OLS point, and the penalised fit is the point of the shaded region on the smallest ring it reaches. A diamond has corners on the axes, so the ring usually meets it where one coefficient is exactly zero.

# The two paths



Ridge shrinks every coefficient smoothly and none reaches zero. Lasso drives them to zero one at a time. The alpha scales differ too: ridge's useful range is in the thousands here, lasso's is between 0.01 and 1.

# Lasso in code

Python Code

```
from sklearn.linear_model import Lasso

lasso = Pipeline([("scale", StandardScaler()), ("lasso", Lasso(alpha=0.1))])
lasso.fit(train[columns], train["vol_next"])

print(rmse(test["vol_next"], lasso.predict(test[columns])))
```

0.21658175865699544

The same pipeline with **Lasso** in the second step scores 0.217, level with ridge.

# Which columns survived

Python Code

```
coefs = lasso.named_steps["lasso"].coef_  
  
for name, b in zip(columns, coefs):  
    if b != 0:  
        print(name, round(b, 3))
```

```
vol_5d 0.129  
vol_10d 0.094  
ret_5d -0.047  
AAPL_vol 0.078  
XOM_vol 0.023
```

# Choosing lasso's alpha

Python Code

```
lasso_grid = {"lasso__alpha": [0.001, 0.01, 0.1, 1]}

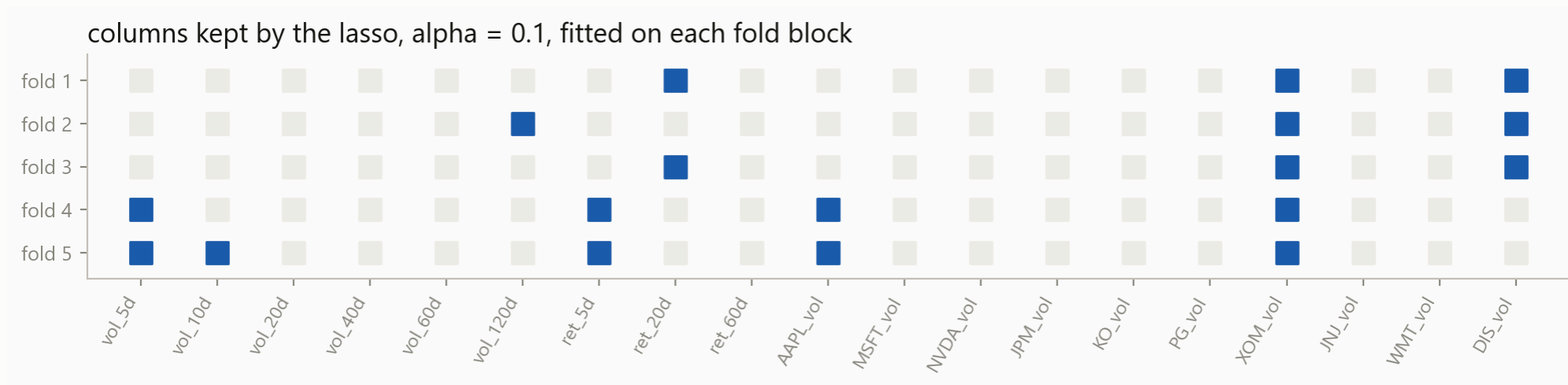
lasso_search = GridSearchCV(lasso, lasso_grid, cv=folds, scoring="neg_root_mean_squared_error")
lasso_search.fit(train[colums], train["vol_next"])

print(lasso_search.best_params_)
print(-lasso_search.best_score_)
```

```
{'lasso__alpha': 0.1}
0.579702888792354
```

This is the same search with a grid on lasso's scale. Alpha 0.1 has the lowest cross-validated error, 0.580 against 0.560 for ridge.

# The zeros are not a verdict



On the early blocks the lasso keeps **XOM\_vol** and **DIS\_vol**; on the later ones, **vol\_5d** and **AAPL\_vol**. Among near-copies it keeps one, and which one depends on the rows.

A zero means the column was not needed **given the others**, on these rows. It does not mean the column carries no information.

# Ridge or lasso

## Ridge

Every column stays in with a small coefficient. Suits a table where many columns each carry a little and are correlated: the weight is spread across near-copies instead of landing on one.

## Lasso

A short list of columns and zeros elsewhere. Suits a table where a few columns matter, or where a compact forecast is wanted. Among near-copies it keeps one, chosen by the rows.

Cross-validate both and compare. Here ridge is slightly ahead, and the two are within the fold spread of each other.

# Elastic net

$$\text{RSS} + \alpha \left[ \rho \sum_j |\beta_j| + \frac{1-\rho}{2} \sum_j \beta_j^2 \right]$$

The elastic net uses both penalties at once. **alpha** sets the total and **l1\_ratio**, the  $\rho$  above, sets the mix: 1 is the lasso, 0 is ridge. It is used when the columns are correlated and a compact model is still wanted.

# Searching two settings at once

Python Code

```
from sklearn.linear_model import ElasticNet

enet = Pipeline([("scale", StandardScaler()), ("enet", ElasticNet())])
enet_grid = {"enet__alpha": [0.01, 0.1, 1], "enet__l1_ratio": [0.1, 0.5, 0.9]}

enet_search = GridSearchCV(enet, enet_grid, cv=folds, scoring="neg_root_mean_squared_error")
enet_search.fit(train[colums], train["vol_next"])
print(enet_search.best_params_, -enet_search.best_score_)
```

```
{'enet__alpha': 0.1, 'enet__l1_ratio': 0.5} 0.5728711001729452
```

Two keys in the grid give three times three combinations, and five folds make forty-five fits. The best mix keeps seven columns.

---

# 6 • Reading the arguments

## PART 6

Which settings matter, which do not, and how to find out.

# Every setting has a default

Python Code

```
Ridge().get_params()
```

```
{'alpha': 1.0,  
 'copy_X': True,  
 'fit_intercept': True,  
 'max_iter': None,  
 'positive': False,  
 'random_state': None,  
 'solver': 'auto',  
 'tol': 0.0001}
```

# When to change what

## Always

`alpha`, the penalty's strength: by cross-validation on a grid in factors of ten. `cv`: `TimeSeriesSplit` for any time series. `scoring`: the error you will report.

## Rarely

`fit_intercept`, an unpenalised intercept: leave it on. `positive`, coefficients of zero or above: when the sign is known. `tol`, when the solver counts as finished: leave it.

## Sometimes

`max_iter`, the passes the lasso solver may take: raise it when the warning appears. `l1_ratio`, the mix: for the elastic net. `n_jobs=-1`, folds in parallel: on your own machine.

## Why only lasso has `max_iter`

`Ridge` has a closed-form solution. `Lasso` and `ElasticNet` are solved by repeated passes over the columns, and the passes can run out.

# The convergence warning

```
ConvergenceWarning: Objective did not converge. You might want to increase
the number of iterations, check the scale of the features or consider
increasing regularisation.
```

The lasso solver ran out of passes before settling. The result is returned anyway, and it is slightly wrong.

## Standardise first

columns on wildly different scales are the usual cause, and the pipeline already handles it

## Raise max\_iter

`Lasso(alpha=0.001, max_iter=10000)`. A very small alpha needs more passes than the default 1,000

---

# 7 • What the coefficients mean now

## PART 7

Penalised coefficients are for forecasting, and what that rules out.

# Prediction, not inference

## Ordinary least squares

Unbiased coefficients with standard errors, so a t-test can ask whether a column has an effect. The econometrics toolkit applies.

## Ridge and lasso

Coefficients shrunk towards zero on purpose. They are biased, they have no standard errors, and there is no `summary()`. They answer one question: what forecast.

To test whether a column matters, fit OLS on a few columns chosen for a reason and read the standard errors. To forecast, penalise.

## What can still be said

A standardised ridge coefficient of 0.079 on `vol_5d` means: **one standard deviation more volatility over the last five days raises the forecast by 0.079 percentage points**, with the other eighteen columns held where they are.

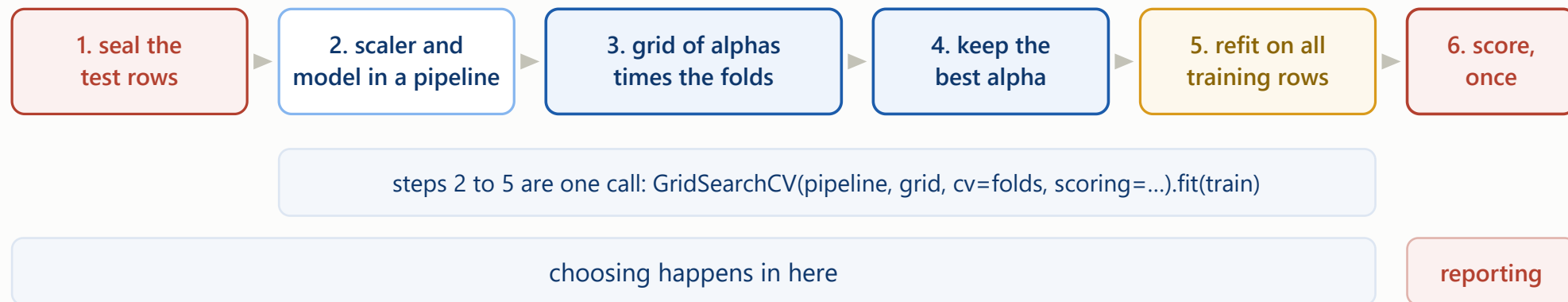
Among near-copies the split is arbitrary: 0.079 on `vol_5d` and 0.061 on `vol_10d` could as well be the other way round. Read related columns together.

## Six forecasts of the same thing

|                                      |                                      |                            |                                          |                                            |                                            |
|--------------------------------------|--------------------------------------|----------------------------|------------------------------------------|--------------------------------------------|--------------------------------------------|
| <b>0.291</b><br>guess the<br>average | <b>0.260</b><br>repeat this<br>month | <b>0.237</b><br>one column | <b>0.248</b><br>nineteen<br>columns, OLS | <b>0.218</b><br>nineteen<br>columns, ridge | <b>0.217</b><br>nineteen<br>columns, lasso |
|--------------------------------------|--------------------------------------|----------------------------|------------------------------------------|--------------------------------------------|--------------------------------------------|

All six are test RMSEs in percentage points. Nineteen columns were worse than one until the coefficients were held small; then they were the first to beat it, with alpha chosen without the test rows.

# The workflow now



The workflow from last time, with the candidates replaced by a grid of alpha values and the model replaced by a pipeline.

# What stays with you

## Many columns overfit

Nineteen columns fitted the training rows better than one and forecast worse. Near-copies get large, opposite coefficients that are noise.

## The penalty needs standardised columns

It charges by coefficient size, and size depends on units. `StandardScaler` fitted on the training rows, then a `Pipeline`.

## GridSearchCV runs the whole search

Every value times every fold, the best mean kept, refitted on all training rows. `best_params_`, `best_score_`, `cv_results_`, `.predict()`.

## The scales are not the same

Ridge's useful alpha was in the thousands, lasso's near 0.1. Read the validation curve rather than reusing a grid.

## A penalty holds coefficients small

Ridge adds  $\alpha \sum \beta_j^2$ , lasso adds  $\alpha \sum |\beta_j|$ . `Ridge(alpha=...)` fits and predicts like `LinearRegression`.

## Alpha is a hyperparameter

Set before `.fit()`, never changed by it. Chosen by cross-validation on the training rows, on a grid in factors of ten.

## Lasso gives exact zeros

A compact model, and the kept set changes with the rows. Ridge spreads weight across correlated columns.

## Forecasting, not inference

Penalised coefficients are biased on purpose and have no standard errors. Fit OLS on a few chosen columns to test an effect.

# Where this goes next

|               |                                                                        |                           |
|---------------|------------------------------------------------------------------------|---------------------------|
| <b>before</b> | fitting a model, and choosing between candidates with cross-validation | linear regression         |
| <b>now</b>    | holding coefficients small, and choosing how small with a grid search  | ridge, lasso, elastic net |
| <b>next</b>   | classification, where the same penalty appears under the name C        | logistic regression       |
| <b>then</b>   | trees and ensembles, with more hyperparameters and the same search     | the same workflow         |

Every later model has settings chosen the way alpha was chosen today: a pipeline, a grid, the folds, and the test rows opened once at the end.

## Before the next session **CASE**

- **Exercises:** building the wide table from the price file, ridge and lasso, standardising inside a pipeline, and reading a grid search.
- **Case Part 6:** the risk report's model gets every column the desk can offer, and a penalty chosen by cross-validation.

Stuck? Ask a friend, ask an AI for a hint rather than an answer, or email [jobo@econ.au.dk](mailto:jobo@econ.au.dk).